



LiquidAI / LFM2.5-1.2B-Thinking



374

Follow

Liquid AI

4.06k

Text Generation Transformers Safetensors 8 languages lfm2 liquid lfm2.5
edge conversational arxiv:2511.23404 License: lfm1.0



Deploy

→ Copy to bucket **NEW**

Use this model



Model card



Files



xet



Community 3

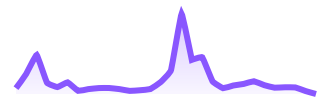
[Try LFM](#) • [Docs](#) • [LEAP](#) • [Discord](#)

LFM2.5-1.2B-Thinking

LFM2.5 is a new family of hybrid models designed for **on-device deployment**. It builds on the LFM2 architecture with extended pre-training and reinforcement learning.

- **Best-in-class performance:** A 1.2B model rivaling much larger models, bringing high-quality AI to your pocket.
- **Fast edge inference:** 239 tok/s decode on AMD CPU, 82 tok/s on mobile NPU. Runs under 1GB of memory with day-one support for llama.cpp, MLX, and vLLM.
- **Scaled training:** Extended pre-training from 10T to 28T tokens and large-scale multi-stage reinforcement learning.

Downloads last month

48,812

Safetensors ⓘ

Model size 1B params

Tensor type BF16 Chat template

Files info

⚡ Inference Providers **NEW**

Text Generation

This model isn't deployed by any Inference Provider.



5

Ask for provider support

🌳 Model tree for LiquidAI/LFM2.5-1...

Base model LiquidAI/LFM2.5-1.2B-Ba...

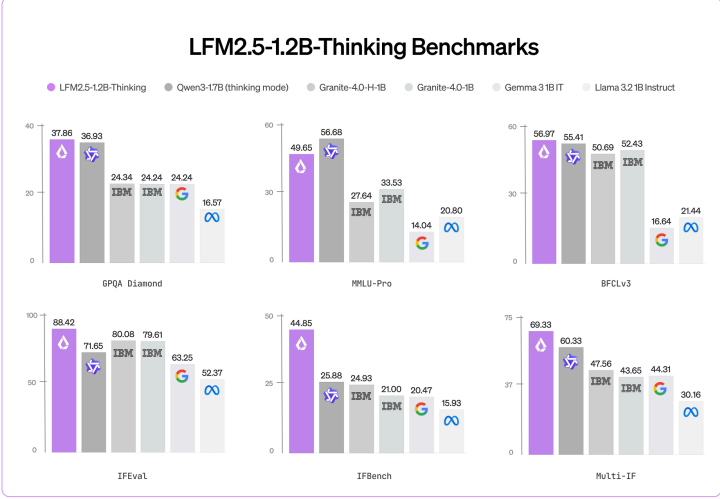
Finetuned (47) [this model](#)

Adapters 1 model

Finetunes 32 models

Quantizations ⚡📄👋 32 models

🏠 Spaces using LiquidAI/LFM2.5... 14



Find more information about LFM2.5-1.2B-Thinking in our [blog post](#).

Model Details

Model	Parameters	Description
LFM2.5-1.2B-Base	1.2B	Pre-trained base model for fine-tuning
LFM2.5-1.2B-Instruct	1.2B	General-purpose instruction-tuned model
LFM2.5-1.2B-Thinking	1.2B	General-purpose reasoning model
LFM2.5-1.2B-JP	1.2B	Japanese-optimized chat model
LFM2.5-VL-1.6B	1.6B	Vision-language model with fast inference
LFM2.5-Audio-1.5B	1.5B	Audio-language model for speech and text I/O

LFM2.5-1.2B-Thinking is a general-purpose text-only model with the following features:

- [LiquidAI/LFM2.5-1.2B-Thinking-Web...](#)
- [sugatoray/LFM2.5-1.2B-Thinking-Web...](#)
- [z4iXsECHS6ND-heightSAI-INSEGCSyX...](#)
- [yassermd/LFM2.5-1.2B-Thinking-We...](#)
- [trixio/local-ai](#)
- [Nirav-Madhani/Meditation](#)
- [Rights4AI/LFM2.5-1.2B-Thinking-Web...](#)
- [krisshattanicole/GEAUXTHNQWebG...](#)

+ 6 Spaces

Collection including LiquidAI/LFM...

LFM2.5 Collection

Collection ... • 14 items • Up... • 168

Paper for LiquidAI/LFM2.5-1.2B-...

LFM2 Technical Report

Paper • 2511.23404 • Publi... • 65

- **Number of parameters:** 1.17B
- **Number of layers:** 16 (10 double-gated LIV convolution blocks + 6 GQA blocks)
- **Training budget:** 28T tokens
- **Context length:** 32,768 tokens
- **Vocabulary size:** 65,536
- **Knowledge cutoff:** Mid-2024
- **Languages:** English, Arabic, Chinese, French, German, Japanese, Korean, Spanish
- **Generation parameters:**
 - temperature: 0.05
 - top_k: 50
 - repetition_penalty: 1.05

Model	Description
<u>LFM2.5-1.2B-Thinking</u>	Original model checkpoint in native format. Best for fine-tuning or inference with Transformers and vLLM.
<u>LFM2.5-1.2B-Thinking-GGUF</u>	Quantized format for llama.cpp and compatible tools. Optimized for CPU inference and local deployment with reduced memory usage.
<u>LFM2.5-1.2B-Thinking-ONNX</u>	ONNX Runtime format for cross-platform deployment. Enables hardware-accelerated inference across diverse environments (cloud, edge, mobile).
<u>LFM2.5-1.2B-Thinking-MLX</u>	MLX format for Apple Silicon. Optimized for fast inference on Mac devices using the MLX framework.

We recommend using it for agentic tasks, data extraction, and RAG. It is not recommended for knowledge-intensive tasks and programming.

Chat Template

LFM2.5 uses a ChatML-like format. See the [Chat Template documentation](#) for details. Example:

```
<|startoftext|><|im_start|>system
You are a helpful assistant trained by Liquid
<|im_start|>user
What is C. elegans?<|im_end|>
<|im_start|>assistant
```

You can use `tokenizer.apply_chat_template()` to format your messages automatically.

Tool Use

LFM2.5 supports function calling as follows:

1. **Function definition:** We recommend providing the list of tools as a JSON object in the system prompt. You can also use the `tokenizer.apply_chat_template()` function with tools.
2. **Function call:** By default, LFM2.5 writes Pythonic function calls (a Python list between `<|tool_call_start|>` and `<|tool_call_end|>` special tokens), as the assistant answer. You can override this behavior by asking the model to output JSON function calls in the system prompt.
3. **Function execution:** The function call is executed, and the result is returned as a "tool"

role.

4. **Final answer:** LFM2 interprets the outcome of the function call to address the original user prompt in plain text.

See the [Tool Use documentation](#) for the full guide.

Example:

```
<|startoftext|><|im_start|>system
List of tools: [{"name": "get_candidate_status", "description": "Get the current status of a candidate."}]
<|im_start|>user
What is the current status of candidate ID 12345?
<|im_start|>assistant
<|tool_call_start|>[get_candidate_status(candidate_id="12345")]
<|im_start|>tool
[{"candidate_id": "12345", "status": "Interviewed"}]
<|im_start|>assistant
The candidate with ID 12345 is currently in the interview stage.
```

 Inference

LFM2.5 is supported by many inference frameworks. See the [Inference documentation](#) for the full list.

Name	Description	Docs	Notebook
Transformers	Simple inference with direct access to model internals.	Link	 Open in Colab
vLLM	High-throughput production deployments with GPU.	Link	 Open in Colab
llama.cpp	Cross-platform inference with CPU offloading.	Link	 Open in Colab

Name	Description	Docs	Notebook
MLX	Apple's machine learning framework optimized for Apple Silicon.	Link	—
LM Studio	Desktop application for running LLMs locally.	Link	—

Here's a quick start example with Transformers:

```
from transformers import AutoModelForCausalLM

model_id = "LiquidAI/LFM2.5-1.2B-Thinking"
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    device_map="auto",
    dtype="bfloat16",
    # attn_implementation="flash_attention_2"
)

tokenizer = AutoTokenizer.from_pretrained(model_id)
streamer = TextStreamer(tokenizer, skip_prompt=True)

prompt = "What is C. elegans?"

input_ids = tokenizer.apply_chat_template(
    [{"role": "user", "content": prompt}],
    add_generation_prompt=True,
    return_tensors="pt",
    tokenize=True,
).to(model.device)

output = model.generate(
    input_ids,
    do_sample=True,
    temperature=0.05,
    top_k=50,
```

```
repetition_penalty=1.05,  
max_new_tokens=512,  
streamer=streamer,  
)
```

 Fine-Tuning

We recommend fine-tuning LFM2.5 for your specific use case to achieve the best results.

Name	Description	Docs	Notebook
CPT (Unsloth)	Continued Pre-Training using Unsloth for text completion.	Link	 Open in Colab
CPT (Unsloth)	Continued Pre-Training using Unsloth for translation.	Link	 Open in Colab
SFT (Unsloth)	Supervised Fine-Tuning with LoRA using Unsloth.	Link	 Open in Colab
SFT (TRL)	Supervised Fine-Tuning with LoRA using TRL.	Link	 Open in Colab
DPO (TRL)	Direct Preference Optimization with LoRA using TRL.	Link	 Open in Colab
GRPO (Unsloth)	GRPO with LoRA using Unsloth.	Link	 Open in Colab
GRPO (TRL)	GRPO with LoRA using TRL.	Link	 Open in Colab

 Performance

Benchmarks

We compared LFM2.5-1.2B-Thinking with relevant sub-2B models on a diverse suite of benchmarks.

Model	GPQA Diamond	MMLU- Pro	IFEval	IFBench	Multi IF
LFM2.5- 1.2B- Thinking	37.86 (± 0.83)	49.65 (± 0.18)	88.42 (± 0.35)	44.85 (± 0.73)	69.35 (± 0.09)
Qwen3- 1.7B (thinking mode)	36.93 (± 2.07)	56.68 (± 1.29)	71.65 (± 0.13)	25.88 (± 0.30)	60.35 (± 0.02)
LFM2.5- 1.2B- Instruct	38.89	44.35	86.23	47.33	60.98
Qwen3- 1.7B (instruct mode)	34.85	42.91	73.68	21.33	56.48
Granite- 4.0-H-1B	24.34	27.64	80.08	24.93	47.50
Granite- 4.0-1B	24.24	33.53	79.61	21	43.65
Gemma 3 1B IT	24.24	14.04	63.25	20.47	44.35
Llama 3.2 1B Instruct	16.57	20.80	52.37	15.93	30.10

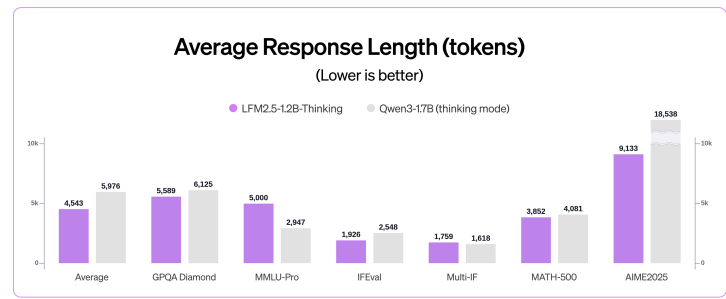
GPQA, MMLU-Pro, IFBench, and AIME25 follow [ArtificialAnalysis's methodology](#). For IFEval and

Multi-IF, we report the average score across strict and loose prompt and instruction accuracies. For BFCLv3, we report the final weighted average score with a custom Liquid handler to support our tool use template.

Based on the same methodology, we report the average score and standard deviation across five runs with temperature=0.6 for thinking models. For instruct models, we report scores using greedy decoding.

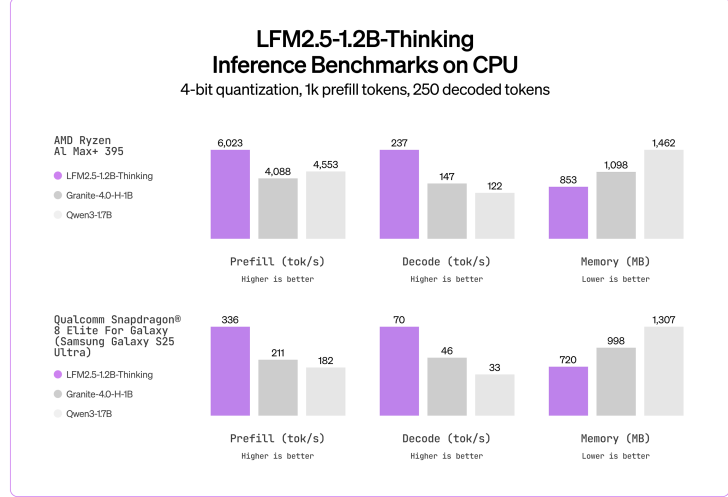
Response length

In comparison with Qwen3-1.7B (thinking mode), it requires fewer output tokens while offering higher overall performance.



Inference speed

LFM2.5-1.2B-Thinking offers extremely fast inference speed on CPUs with a low memory profile compared to similar-sized models.



In addition, we are partnering with AMD, Qualcomm, Nexa AI, and FastFlowLM to bring the LFM2.5 family to NPUs. These optimized models are available through our partners, enabling highly efficient on-device inference.

We report the following numbers with 1K prefill and 100 decode tokens:

Device	Inference	Framework	Model	Prefill (tok/s)
AMD Ryzen AI 395+	NPU	FastFlowLM	LFM2.5-1.2B-Thinking	1487
AMD Ryzen AI 9 HX 370	NPU	FastFlowLM	LFM2.5-1.2B-Thinking	1487
AMD Ryzen AI 9 HX 370	CPU	llama.cpp (Q4_0)	LFM2.5-1.2B-Thinking	2975
Qualcomm Snapdragon® X Elite	NPU	NexaML	LFM2.5-1.2B-Thinking	2590

Device	Inference	Framework	Model	Pre (tok
Qualcomm Snapdragon® Gen4 (ROG Phone9 Pro)	NPU	NexaML	LFM2.5-1.2B-Thinking	439.5
Qualcomm Dragonwing IQ9 (IQ-9075) (IoT)	NPU	NexaML	LFM2.5-1.2B-Thinking	214.5
Qualcomm Snapdragon® Gen4 (Samsung Galaxy S25 Ultra)	CPU	llama.cpp (Q4_0)	LFM2.5-1.2B-Thinking	335

LFM2.5-1.2B-Thinking excels at long-context inference. For example, on AMD Ryzen™ NPUs with FastFlowLM, decoding throughput sustains ~52 tok/s at 16K context and ~46 tok/s even at the full 32K context, indicating robust long-context scalability. For more details on longer context benchmarks on AMD Ryzen™ NPUs with FastFlowLM, please review these [here](#).

These capabilities unlock new deployment scenarios across various devices, including vehicles, mobile devices, laptops, IoT devices, and embedded systems.

Contact

- Got questions or want to connect? [Join our Discord community](#).

- If you are interested in custom solutions with edge deployment, please contact [our sales team](#).

Citation

```
@article{liquidAI2026thinking,
  author = {Liquid AI},
  title = {LFM2.5-1.2B-Thinking: On-Device Reasoning},
  journal = {Liquid AI Blog},
  year = {2026},
  note = {Summary: liquid.ai/blog/lfm2-5-1-2b-thinking}
```

```
@article{liquidai2025lfm2,
  title={LFM2 Technical Report},
  author={Liquid AI},
  journal={arXiv preprint arXiv:2511.23404},
  year={2025}
}
```